York University

Fall 2026 Course Project: Stage 1

AI News Analysis Agent

Nicole Van

219563162

EECS 3311 A - Software Design

Dr. Song Wang

October 2, 2026

1. Project Overview

1.1 Problem and Motivation

Keeping up with current events has become increasingly challenging due to the large volume of news available from many different sources. The same event may be reported multiple times, often with different perspectives, details, or potential biases (e.g., political, corporate, public). This makes it difficult for users to determine what information is accurate and consistent across sources. While traditional RSS readers can collect and organize articles in one place, they generally do not analyze the content. Thus, this project proposes an *AI News Analysis Agent* that extends beyond simple news aggregation by grouping related stories, comparing coverage across sources, and extracting important information to provide users with a clearer understanding of current events.

1.2 Target Users

- **Citizens:** People who want to be informed about the current events in the world around them without needing to spend a lot of time manually checking/reading multiple news sources.
- **Journalists:** People who must quickly find and track new stories, verify information, and inform the public.
- **Industry Analysts:** People who must monitor specific topics, trends, and developments to provide insights and recommendations.

1.3 Agent Description

The AI News Analysis Agent is a Java based system that integrates conventional software components with AI to go beyond simply retrieving and summarizing news sources. By dynamically determining the steps and tools needed to fulfil a user's request, this agent will be able to retrieve relevant articles, identify related or duplicate stories, cluster coverage by event, extract key entities, compare reporting across sources, and organize information into summaries and timelines. Together, these capabilities allow the agent to provide users with a consolidated analysis of available news coverage.

1.4 AI/LLM Model(s) and Architecture

The system will integrate the AI agent with a locally hosted big language model using the Java framework LangChain4j. Ollama will provide the local LLM runtime and allow the project to operate without using any paid cloud LLM API. In terms of the specific open-source model, this will be selected during implementation.

Architecture:

Java Application → LangChain4j → Ollama → Local LLM

LangChain4j interactions:

- Used for LLM communication
- Used for AI agent/tool integration
- Used for tool calling
- Used for structuring AI responses
- Used for embeddings where required for the article analysis strategies

---

2. Detailed Feature Specifications

F01 — Topic Creation:

The user enters a topic name they are interested in through the GUI. The controller will create the Topic object and store it (AI Involvement: Deterministic).

F02 — Article Retrieval:

The user selects a topic, then the agent will retrieve articles from external news APIs using the topic's generated queries. It parses through the responses and converts them into a displayable Article object for the User. (AI Involvement: Deterministic).

F03 — Article Categorization:

The agent will analyze each retrieved article through its available tools and assign each article to predefined categories (politics, sports, technology, etc.). The assigned category appears as a tag in the article viewer and can be used by the user to filter the feed. (AI Involvement: AI-based).

F04 — Article Summarization:

The agent will analyze the articles through its available tools and generate a concise summary of the entire article. This summarized version is displayed in the GUI. (AI Involvement: AI-based).

F05 — Duplicate-Story Detection:

The agent will analyze the articles through its available tools and automatically compute similarities between each one. (AI Involvement: AI-based).

F06 — Event Clustering:

The agent will group similar articles into a single event cluster using embeddings. (AI Involvement: Hybrid).

F07 — Entity Extraction:

The agent will analyze the articles through its available tools and extract people, organizations, and locations from the text. Extracted entities are highlighted in the article viewer and are clickable by the user to filter related articles across the feed. (AI Involvement: AI-based).

F08 — Timeline Generation:

The user selects a topic and opens the timeline tab. The agent will then retrieve all associated EventCluster objects and use its available tools to arrange and narrate them chronologically. The result is rendered into a timeline in the GUI. (AI Involvement: Hybrid).

F09 — Multiple Source Comparison:

The user can select an EventCluster, then select an option to compare sources. The agent will use its available tools to autonomously call on each article to compare coverage. A structured comparison report will be generated and displayed in the comparison panel. (AI Involvement: AI-based).

F10 — Claim Extraction:

The agent will use its available tools to extract factual claims made in an article for further analysis and fact checking. (AI Involvement: AI-based).

F11 — Source Linking:

The agent will link analysis results (summaries, entities, claims, etc.) back to their original articles. Clicking on any analysis element in the GUI will open the original source article in the viewer with its URL available for external access. (AI Involvement: Deterministic).

F12 — Topic Tracking:

The user can toggle a track option on a topic. The agent will automatically retrieve new articles for tracked topics and notify the GUI when new content arrives. (AI Involvement: Hybrid).

F13 — Personalized Feed:

The agent will generate a personalized feed by combining the user's tracked topics, saved articles, and search history. The feed is ranked by relevance and displayed on the GUI. (AI Involvement: Hybrid).

F14 — Saved Articles:

The user can click on a bookmark icon to save and remove articles for later access. (AI Involvement: Deterministic).

F15 — Natural-Language News Queries:

The user types a question into the search bar. The agent will embed the query and retrieve relevant articles through its available tools and pass them as context. The agent will then generate an answer and display it on the GUI with sources. (AI Involvement: AI-based).

F16 — Daily Briefing Generation:

The agent will retrieve the day's top EventCluster objects as well as personalized preferences, then use its available tools to generate a summarized briefing. The result is displayed on the GUI. (AI Involvement: Hybrid).

F17 — Search History:

The agent stores and retrieves previous searches and natural-language queries made by the user. (AI Involvement: Deterministic).

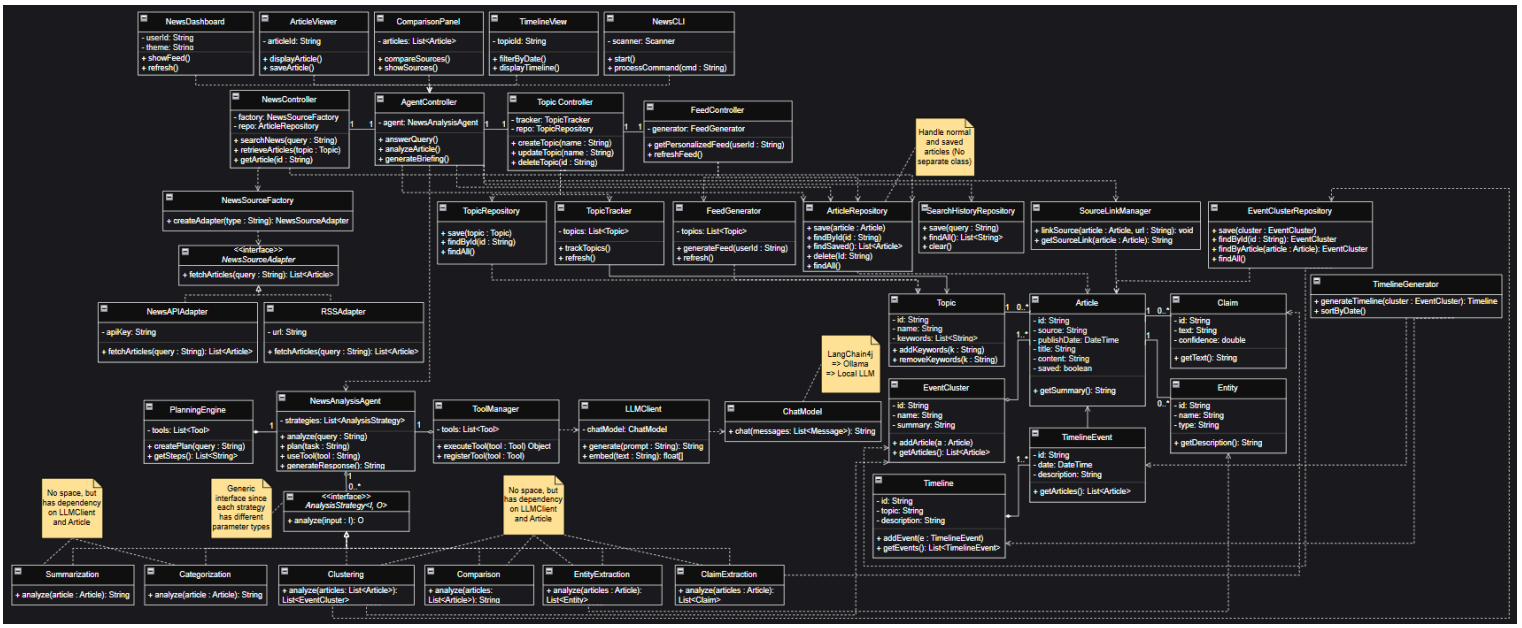
---

3. System Architecture

Important Note:

All UML diagrams below are created in https://app.diagrams.net/ from the EECS3311-3-UML lecture slides (slide 8). Links to each diagram are provided below.

3.1 UML Class Diagram



Link to Diagram: https://drive.google.com/file/d/1fKmoXcZUIogxGbdkRR_2P1oGUATD1qAJ/view?usp=sharing

3.2 Design Patterns

DP01 — Strategy:

The AnalysisStrategy provides a common generic interface for different AI analysis classes (i.e., Summarization, Categorization, Clustering, Comparison, EntityExtraction, ClaimExtraction). The agent can select the appropriate strategy depending on the task. This design pattern is appropriate because it gives an easy way to add new analyses without needing to modify the agent. As well, each analysis is independently testable. Without it, the agent would need large blocks of if-else statements which makes it harder to extend.

DP02 — Observer:

The observer pattern solves the problem of keeping the GUI synchronized when tracked topics retrieve new articles in the background. This design pattern is appropriate because UI components can react to the same event independently of each other, while staying separate from the GUI.

DP03 — Factory Method:

The factory method pattern addresses the need to support multiple news sources without letting the rest of the system know which concrete adapter to instantiate. The participating classes are NewsSourceFactory, NewsSourceAdapter (interface), NewsAPIAdapter, and RSSAdapter. This design pattern is appropriate because adding a new provider requires a new adapter and an update to the factory's creation logic without requiring the rest of the application/system to depend directly on the new provider.

DP04 — Adapter:

The adapter pattern solves the problem of different news providers possibly having different APIs and response formats when the internal system expects an "Article" representation. The participating classes are NewsSourceAdapter, NewsAPIAdapter, RSSAdapter, and Article. Each adapter translates its provider's response into the Article representation and shields the rest of the system from provider specific details. This design pattern is appropriate because the domain layer only sees Article objects regardless of how many providers are added. Without this, each provider's formats would need to be handled, causing tight coupling.

DP05 — Facade:

The facade pattern addresses the complexity of the AI subsystem. The AgentController provides a simplified interface to the more complicated AI subsystem. Instead of having the GUI directly interacting with the PlanningEngine, ToolManager, NewsAnalysisAgent, and LLMClient, it can simply call AgentController.answerQuery(...). This design pattern is appropriate because the GUI and CLI stay separate from the internal agent complexity. Without this, every GUI action would need to wire together the planning, tools, LangChain4j services and the underlying analysis components, causing fragile coupling.

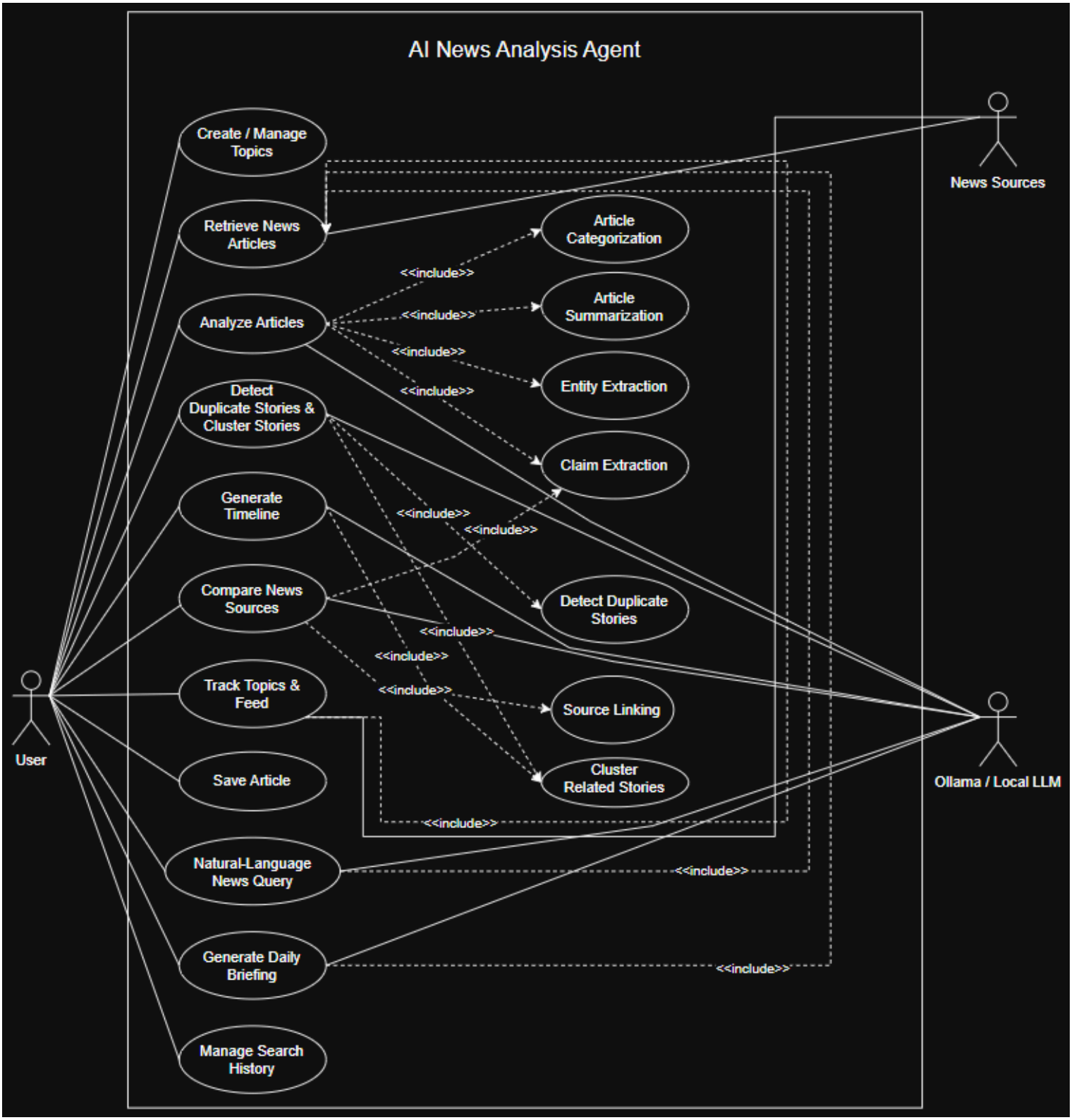
DP06 — MVC:

The MVC pattern helps to separate the user interface from application logic and data. This means that domain objects can be reused across the GUI and CLI without affecting anything else. The participating classes are shown below:

- **Model:** Article, Topic, EventCluster, Timeline, TimelineEvent, Claim, Entity
- **View:** NewsDashboard, ArticleViewer, ComparisonPanel, TimelineView, NewsCLI
- **Controller:** NewsController, AgentController, TopicController, FeedController

The "Model" represents domain states and rules. The "View" renders models and forwards user actions. The "Controller" handles the input, invokes agent services, and updates the models. This design pattern is appropriate because it provides a clear separation. Without this, all the logic and data would be tangled, which makes it harder to test.

3.3 Use-Case Diagram



Link to Diagram: https://drive.google.com/file/d/1z4CGwGN0o2oePlOG1IictK7H_20-9j4X/view?usp=sharing

3.4 Detailed Use-Case Descriptions

UC01 — Create and Manage Topics:

- **Actor(s):** User
- **Goal:** Create, modify, or delete topics that the user is interested in.
- **Preconditions:** The user has access to the system.
- **Trigger:** The user chooses to create or manage a topic.
- **Main Success Scenario:** The user enters or modifies a topic. The system then updates and saves the topic information.

- **Alternative/Exception Flows:** If the topic information is invalid or cannot be saved, then the system will display an error.
- **Postconditions:** The topic is successfully created, updated, or deleted.
- **Related Feature(s):** F01 — Topic Creation

UC02 — Retrieve News Articles:

- **Actor(s):** User, News Sources
- **Goal:** Retrieve relevant news articles from available sources.
- **Preconditions:** In addition to the news sources being accessible, a search topic or query must also be available.
- **Trigger:** The user requests news articles.
- **Main Success Scenario:** The system sends a query to the available news sources. Once received, matching articles are retrieved and presented to the user.
- **Alternative/Exception Flows:** If a source is unavailable or returns no results, then the system will either continue with available sources, or inform the user.
- **Postconditions:** The retrieved articles are available for viewing and further analysis.
- **Related Feature(s):** F02 — Article Retrieval

UC03 — Analyze Articles:

- **Actor(s):** User, Ollama / Local LLM
- **Goal:** Analyze an article and extract useful information from its content.
- **Preconditions:** An article should be available for analysis and the local LLM should be accessible.
- **Trigger:** The user requests analysis of an article.
- **Main Success Scenario:** After user request, the system will categorize and summarize the article, extract relevant entities and claims, then present the results to the user.
- **Alternative/Exception Flows:** If the analysis fails, then the system will report the error and return an available partial result.
- **Postconditions:** The article has generated analysis results.
- **Related Feature(s):** F03 — Article Categorization, F04 — Article Summarization, F07 — Entity Extraction, F10 — Claim Extraction

UC04 — Detect Duplicate Stories and Cluster Stories:

- **Actor(s):** User, Ollama / Local LLM
- **Goal:** Identify duplicate articles and group related articles into event clusters.
- **Preconditions:** Multiple articles should be available for comparison.
- **Trigger:** The user requests duplicate detection or story clustering.
- **Main Success Scenario:** After user request, the system will compare the articles, identify duplicate or highly similar stories, then group the related articles into clusters.
- **Alternative/Exception Flows:** If insufficient articles are available, the system will inform the user that this function cannot be performed.
- **Postconditions:** Duplicate relationships and/or event clusters are available for further analysis.
- **Related Feature(s):** F05 — Duplicate-Story Detection, F06 — Event Clustering

UC05 — Generate Timeline:

- **Actor(s):** User, Ollama / Local LLM
- **Goal:** Generate a chronological timeline of events from related news coverage.
- **Preconditions:** Relevant articles or event clusters are available.

- **Trigger:** The user requests a timeline.
- **Main Success Scenario:** After user request, the system will analyze the relevant stories, identify events and dates, order them chronologically, and display the resulting timeline.
- **Alternative/Exception Flows:** If the date or a sufficient amount of information cannot be extracted, then the system will either generate a partial timeline or report that a timeline cannot be created.
- **Postconditions:** A chronological timeline is presented to the user.
- **Related Feature(s):** F08 — Timeline Generation

UC06 — Compare News Sources:

- **Actor(s):** User, Ollama / Local LLM
- **Goal:** Compare how different news sources report the same story.
- **Preconditions:** Articles covering a related story should be available from multiple news sources.
- **Trigger:** The user requests a source comparison.
- **Main Success Scenario:** After user request, the system will compare the articles, extract relevant claims, identify the differences, and link the compared sources.
- **Alternative/Exception Flows:** If there are insufficient sources available, then the system will inform the user that a comparison cannot be completed.
- **Postconditions:** A source comparison is presented with links to the relevant sources.
- **Related Feature(s):** F09 — Multiple Source Comparison, F10 — Claim Extraction, F11 — Source Linking

UC07 — Track Topics and Feed:

- **Actor(s):** User, News Sources
- **Goal:** Allow the user to follow topics and receive a feed of relevant news.
- **Preconditions:** The user has at least one topic available to track.
- **Trigger:** The user chooses to track a topic or refreshes their feed.
- **Main Success Scenario:** The system will monitor the user's selected tracked topics. The system will retrieve relevant articles, and update the user's personalized feed.
- **Alternative/Exception Flows:** If no new articles are available or a news source is unavailable, then the system will keep the existing feed and inform the user if necessary.
- **Postconditions:** The user's selected tracked topics and personalized feed are updated.
- **Related Feature(s):** F12 — Topic Tracking, F13 — Personalized Feed

UC08 — Save Articles:

- **Actor(s):** User
- **Goal:** Save an article for later access.
- **Preconditions:** An article is available to the user.
- **Trigger:** The user selects the option to save a particular article.
- **Main Success Scenario:** The system will mark the article as "saved" and store the updated article information.
- **Alternative/Exception Flows:** If the article cannot be saved for whatever reason, the system will display an error.
- **Postconditions:** The article is available as a saved article.
- **Related Feature(s):** F14 — Saved Articles

UC09 — Natural-Language News Query:

- **Actor(s):** User, Ollama / Local LLM
- **Goal:** Allow the user to ask questions about the news using natural language.

- **Preconditions:** The system and local LLM are available.
- **Trigger:** The user types and submits a natural-language news question.
- **Main Success Scenario:** After the user query is submitted, the agent will interpret the query, determine the required steps and tools, retrieve relevant information, analyze it, and return a response to the user.
- **Alternative/Exception Flows:** If the query cannot be understood or insufficient information is found, then the system will ask the user for clarification or report that no suitable answer was found.
- **Postconditions:** The user receives a response to their news query.
- **Related Feature(s):** F15 — Natural-Language News Queries

UC10 — Generate Daily Briefing:

- **Actor(s):** User, Ollama / Local LLM
- **Goal:** Generate a concise briefing of important recent news.
- **Preconditions:** In addition to relevant news articles being available, the local LLM should be accessible.
- **Trigger:** The user requests a daily briefing.
- **Main Success Scenario:** After user request, the system will retrieve relevant articles, group related stories, summarize the important information, and present the user with a consolidated briefing.
- **Alternative/Exception Flows:** If insufficient news is available, the system will generate a partial briefing or inform the user.
- **Postconditions:** A consolidated daily news briefing is presented to the user.
- **Related Feature(s):** F16 — Daily Briefing Generation

UC11 — Manage Search History:

- **Actor(s):** User
- **Goal:** Allow the user to view and manage their previous searches.
- **Preconditions:** The system is available and the search history can be accessed.
- **Trigger:** The user opens or manages their search history.
- **Main Success Scenario:** The system will retrieve the user's previous searches and allow the user to view or clear them.
- **Alternative/Exception Flows:** If no search history exists, the system will inform the user that there are no previous searches.
- **Postconditions:** The user's search history is displayed or updated.
- **Related Feature(s):** F17 — Search History

3.5 Sequence Diagrams

Important Note:

The graphical representation of a UML sequence diagram is made for only three of the eight as examples of how the text-based sequence will be translated over.

SD01 — Article Retrieval and Analyzation:

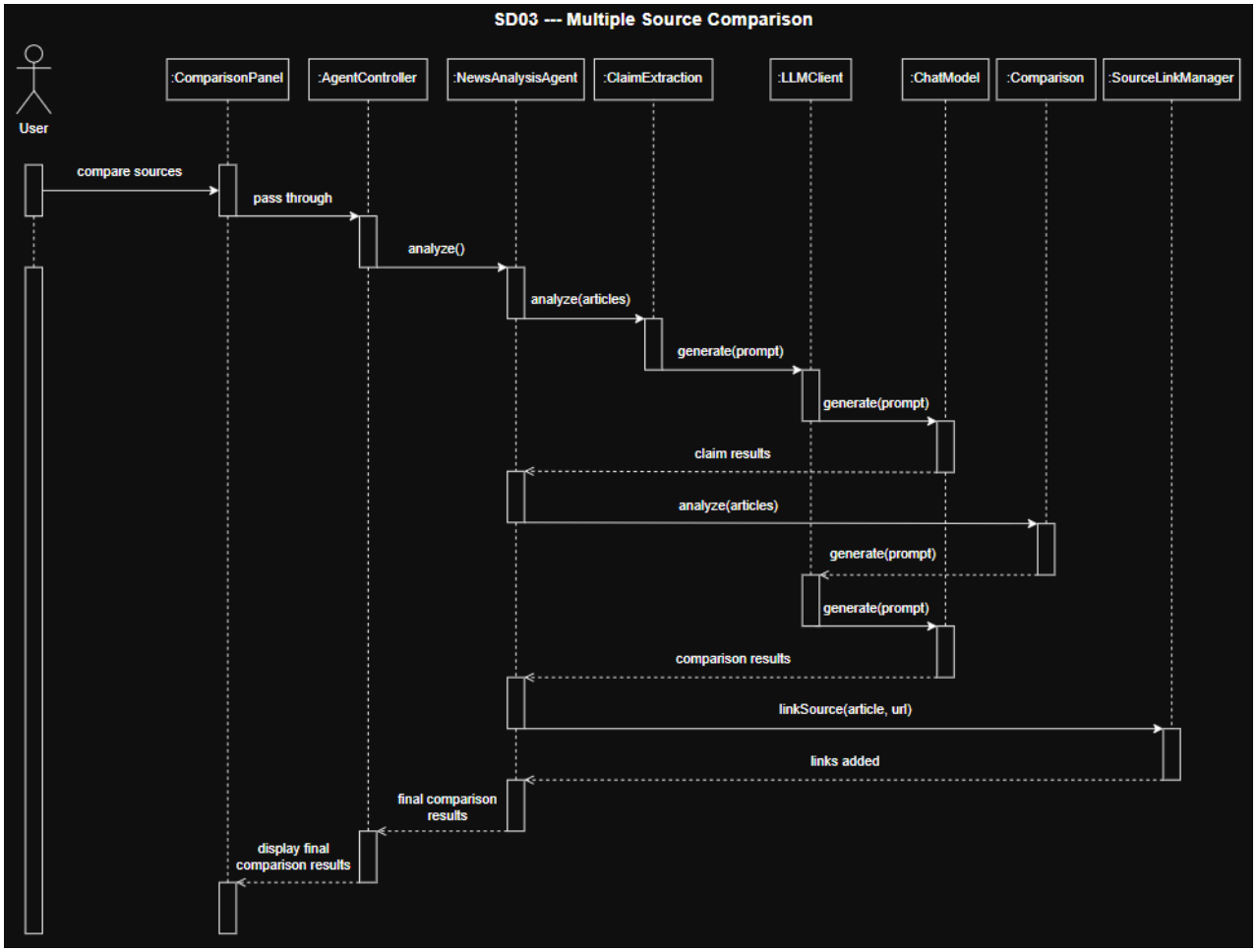
- User → NewsDashboard → NewsController → NewsSourceFactory → NewsSourceAdapter → ArticleRepository → NewsDashboard → AgentController → NewsAnalysisAgent → AnalysisStrategy/Strategy → LLMClient → LangChain4j → Ollama/Local LLM → Analysis Results → NewsDashboard

SD02 — Duplicate-Story Detection and Cluster Events:

- User → AgentController → NewsAnalysisAgent → Clustering → ArticleRepository → LLMClient → LangChain4j → Ollama Embedding → Clustering → EventClusterRepository → NewsDashboard

SD03 — Multiple Source Comparison:

- User → ComparisonPanel → AgentController → NewsAnalysisAgent → ClaimExtraction → LLMClient → LangChain4j → Ollama → Claim Results → Comparison → LLMClient → LangChain4j → Ollama → Comparison Results → SourceLinkManager → ComparisonPanel



SD04 — Topic Tracking and Feed Updates:

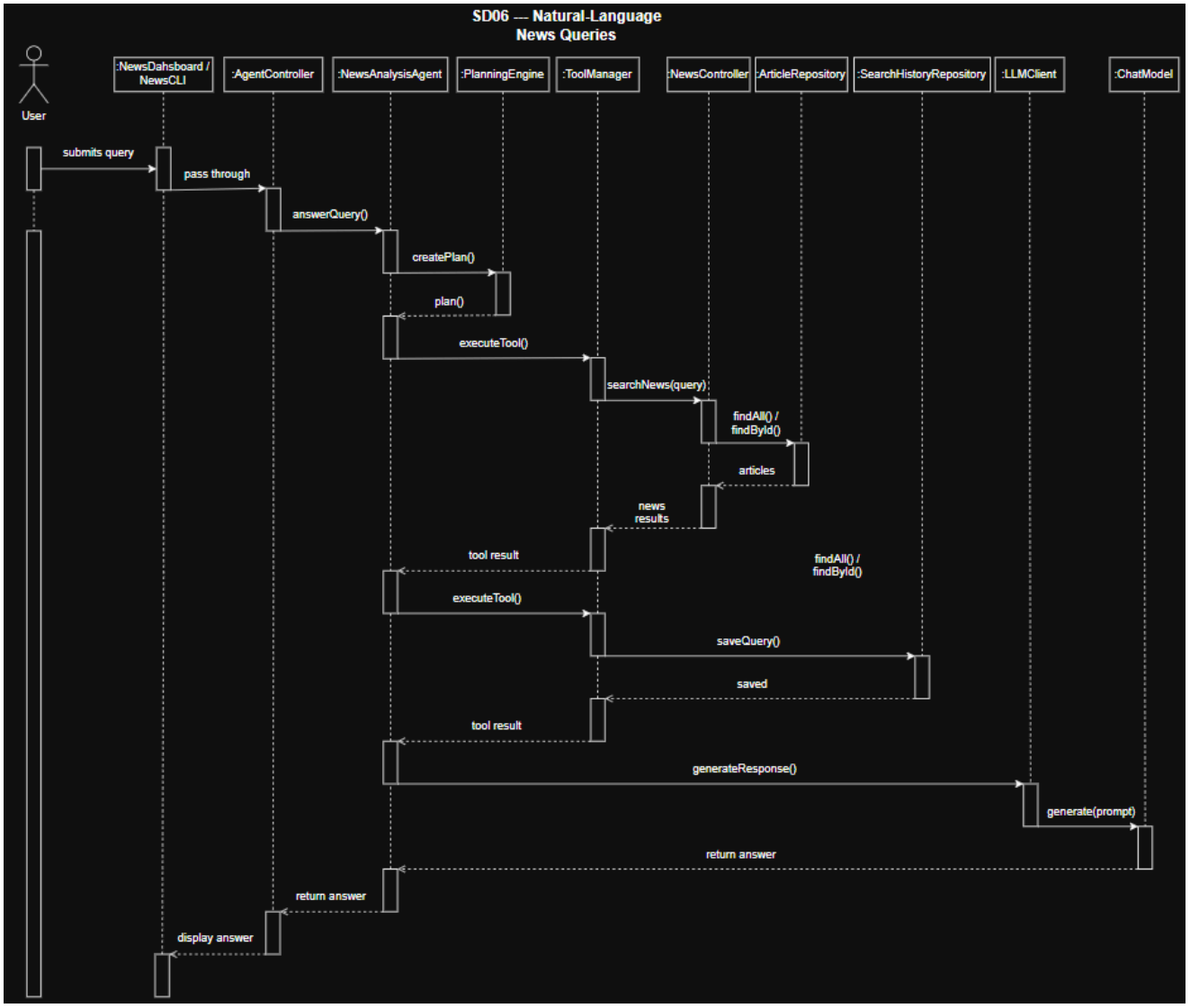
- User → TopicController → TopicRepository → Topic
- User → TopicController → TopicTracker → NewsController → NewsSourceFactory → NewsSourceAdapter → ArticleRepository
- TopicController → FeedController → FeedGenerator → NewsDashboard

SD05 — Timeline Generation:

- User → TimelineView → AgentController → NewsAnalysisAgent → ArticleRepository → Clustering → LLMClient → LangChain4j → Ollama Embedding → EventClusterRepository → EventCluster → TimelineGenerator → Timeline → TimelineEvent → TimelineView

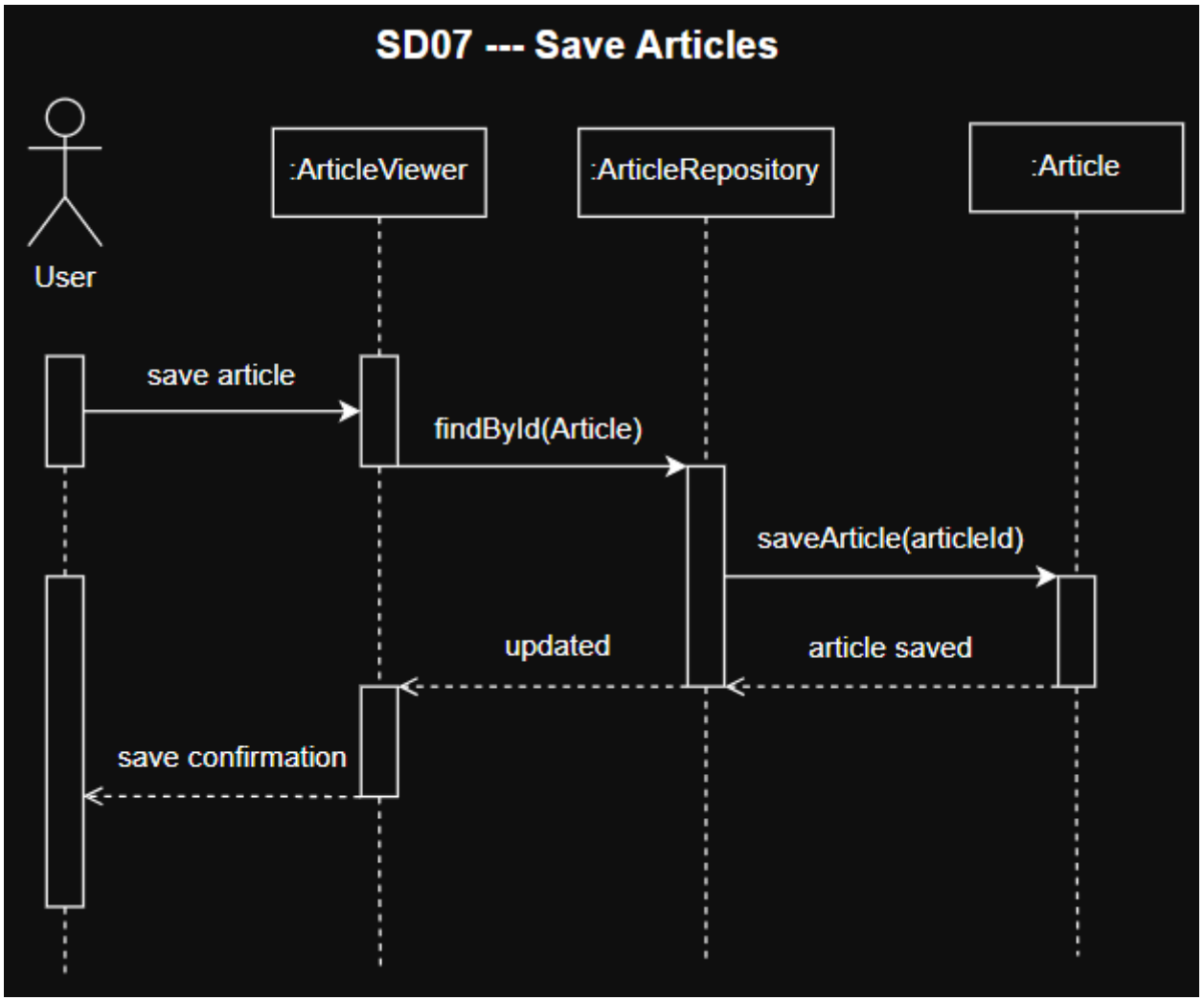
SD06 — Natural-Language News Queries:

- User → NewsDashboard/NewsCLI → AgentController → NewsAnalysisAgent → PlanningEngine → ToolManager → NewsController/ArticleRepository/SearchHistoryRepository → LLMClient → ChatModel → NewsAnalysisAgent → AgentController → NewsDashboard/NewsCLI



SD07 — Saved Articles:

- User → ArticleViewer → ArticleRepository → Article



SD08 — Daily Briefing Generation:

- User → AgentController → NewsAnalysisAgent → ArticleRepository → Clustering → LLMClient → LangChain4j → Ollama Embedding → EventCluster → Summarization → LLMClient → LangChain4j → Ollama → Daily Briefing Results → NewsDashboard

Link to Diagrams:

https://drive.google.com/file/d/1ySGVUQOHNUDBbunWlPk7tUmFNur3OeiZ/view?usp=sharing

---

4. Feature-to-Design Traceability

Feature	Description	Type	Use Case	Classes	Key Methods	Seq. Diagram	Design Patterns
F01	Topic Creation	D	UC01	NewsDashboard, TopicController, TopicRepository, Topic	showFeed() createTopic(), save(), refresh()	SD04	Strategy, MVC
F02	Article Retrieval	D	UC02	NewsController, NewsSourceFactory, NewsSourceAdapter, NewsSourceAPIAdapter, RSSAdapter,	retrieveArticles(), createAdapter(), fetchArticles(), save()	SD01	Adapter, Factory, MVC

				ArticleRepository, Article			
F03	Article Categorization	AI	UC03	AgentController, NewsAnalysisAgent, Categorization, Article, LLMClient	analyzeArticle(), analyze(), generate()	SD01	Strategy, Facade
F04	Article Summarization	AI	UC03	AgentController, NewsAnalysisAgent, Summarization, Article, LLMClient	analyzeArticle(), analyze(), generate()	SD01	Strategy, Facade
F05	Duplicate Detection	AI	UC04	AgentController, NewsAnalysisAgent, Clustering, ArticleRepository, Article, LLMClient	analyze(), findAll(), embed()	SD02	Strategy
F06	Event Clustering	H	UC04	AgentController, NewsAnalysisAgent, Clustering, EventClusterRepository, EventCluster, Article, ArticleRepository	analyze(), findAll(), addArticle(), save()	SD02	Strategy, Observer
F07	Entity Extraction	AI	UC03	AgentController, NewsAnalysisAgent, EntityExtration, Entity, Article, LLMClient	analyzeArticle(), analyze(), generate()	SD01	Strategy, Facade
F08	Timeline Generation	H	UC05	TimelineView, AgentController, NewsAnalysisAgent, ArticleRepository, Clustering, EventClusteringRepository, TimelineGenerator, Timeline, TimelineEvent	generateTimeline-(), findAll(), analyze(), save(), sortByDate(), addEvent()	SD05	Strategy, MVC
F09	Multi-Source Comparison	AI	UC06	ComparisonPanel, AgentController, NewsAnalysisAgent, Comparison, ClaimExtraction, LLMClient, SourceLinkManager, Article	compareSources(), analyze(), generate(), getSourceLink()	SD03	Strategy, Facade, MVC
F10	Claim Extraction	AI	UC06	AgentController, NewsAnalysisAgent, ClaimExtraction, Claim, Article, LLMClient	analyze(), generate()	SD03	Strategy, Facade
F11	Source Linking	D	UC06	ComparisonPanel, SourceLinkManager, Article	linkSource(), getSourceLink()	SD03	MVC
F12	Topic Tracking	H	UC07	TopicController, TopicTracker, TopicRepository, NewsController, NewsSourceFactory,	createTopic(), updateTopic(), deleteTopic(), trackTopic(), refresh(),	SD04	Observer, Factory

				NewsSourceAdapter, ArticleRepository, NewsDashboard	retrieveArticles(), createAdapter(), fetchArticles(), save()		
F13	Personalized Feed	H	UC07	FeedController, FeedGenerator, TopicRepository, ArticleRepository, SearchHistoryRepository, NewsDashboard	getPersonalizedFeed(), generateFeed(), refreshFeed(), findAll()	SD04	Observer, MVC
F14	Saved Articles	D	UC08	ArticleViewer, ArticleRepository, Article	saveArticle(), save(), removeSave(), findById(), findSaved()	SD07	MVC
F15	Natural Language Query	AI	UC09	NewsDashboard, NewsCLI, AgentController, NewsAnalysisAgent, PlanningEngine, ToolManager, LLMClient, NewsController, ArticleRepository	start(), processCommand(), answerQuery(), analyze(), plan(), createPlan(), executeTool(), searchNews(), save(), generate(), chat()	SD06	Facade, Strategy
F16	Daily Brief Generation	H	UC10	NewsDashboard, AgentController, NewsAnalysisAgent, ArticleRepository, Clustering, Summarization, LLMClient	generateBriefing(), findAll(), analyze(), getArticles(), generate()	SD08	Facade, Strategy
F17	Search History	D	UC11	NewDashboard, NewsCLI, AgentController, SearchHistoryRepository	answerQuery(), save(), findAll(), clear()	SD06	MVC

---

5. Feature Implementation to Realization

F01 — Topic Creation:

- **Related Use Case:** UC01 — Create and Manage Topics
- **Related Sequence Diagram:** SD04 — Topic Tracking and Feed Updates
- **Classes Involved:**
  - NewsDashboard — provides the GUI where the user can create and manage topics
  - TopicController — coordinates the topic creation request from the user interface
  - TopicRepository — stores and retrieves the Topic objects
  - Topic — represents a news topic
- **Important Methods:**
  - showFeed()
  - createTopic()

    - save()
    - refresh()
- **Execution:** The user enters and submits a topic in the NewsDashboard. The request is forwarded to TopicController which creates a Topic object containing the name. This is then passed to TopicRepository.save() for storage. Once the topic is created, the NewsDashboard can be refreshed to display the updated topic information.

F02 — Article Retrieval:

- **Related Use Case:** UC02 — Retrieve News Articles
- **Related Sequence Diagram:** SD01 — Article Retrieval and Analyzation
- **Classes Involved:**
    - NewsController — coordinates the article retrieval from external sources
    - NewsSourceFactory — creates the appropriate source adapters based on the selected provider
    - NewsSourceAdapter — common interface for all the news source providers
    - NewsAPIAdapter — adapts the News API's response format into the system's Article representation
    - RSSAdapter — adapts the RSS feed data into the system's Article representation
    - ArticleRepository — stores and retrieves the news articles
    - Article — represents a retrieved news article and its associated information
- **Important Methods:**
    - retriveArticles()
    - createAdapter()
    - fetchArticles()
    - save()
- **Execution:** When articles need to be retrieved, the NewsController will request the appropriate source adapter from NewsSourceFactory. The factory will then create a NewsAPIAdapter or RSSAdapter through the NewsSourceAdapter interface depending on the selected source. The adapter will then retrieve news articles using the fetchArticles() method and will convert the provider specific response into the system's Article representation. These retrieved articles are then stored using the ArticleRepository.save() method.

F03 — Article Categorization:

- **Related Use Case:** UC03 — Analyze Articles
- **Related Sequence Diagram:** SD01 — Article Retrieval and Analyzation
- **Classes Involved:**
    - AgentController — provides the application-level interface for requesting AI analysis
    - NewsAnalysisAgent — coordinates the analysis process and selects the appropriate analysis strategy
    - AnalysisStrategy — a generic interface for each analysis strategy
    - Categorization — the strategy that uses AI to categorize the articles
    - LLMClient — provides access to the language model used for AI processing
    - Article — the article content that is analyzed
- **Important Methods:**
    - analyzeArticle()
    - analyze()
    - generate()
- **Execution:** After an article is retrieved, the AgentController can request that the NewsAnalysisAgent should analyze the article. The agent selects the Categorization analysis strategy through the generic

AnalysisStrategy interface. Through the Categorization strategy, relevant article information is sent to LLMClient.generate() which communicates the configured language model to determine the appropriate category. The resulting category is then returned through the agent to the user interface.

F04 — Article Summarization:

- **Related Use Case:** UC03 — Analyze Articles
- **Related Sequence Diagram:** SD01 — Article Retrieval and Analyzation
- **Classes Involved:**
  - AgentController — provides the application-level interface for requesting article analysis
  - NewsAnalysisAgent — coordinates the analysis and selects the appropriate strategy
  - AnalysisStrategy — a generic interface for each analysis strategy
  - Summarization — condenses each article into a summary
  - LLMClient — sends prompts to the language model and returns generated results
  - Article — the article content that is summarized
- **Important Methods:**
  - analyzeArticle()
  - analyze()
  - generate()
- **Execution:** The AgentController will request an article analysis from the NewsAnalysisAgent. The agent then selects the Summarization strategy through the AnalysisStrategy interface. This strategy will then pass the article content to LLMClient.generate() which sends the prompt to the language model, and receives back a generated summary. The summary is then returned through the agent to the user interface.

F05 — Duplicate-Story Detection:

- **Related Use Case:** UC04 — Detect Duplicate Stories and Cluster Stories
- **Related Sequence Diagram:** SD02 — Duplicate-Story Detection and Cluster
- **Classes Involved:**
  - NewsAnalysisAgent — coordinates the analysis required to identify similar articles
  - AnalysisStrategy — a generic interface for each analysis strategy
  - Clustering — performs similarity and clustering analysis on groups of articles
  - LLMClient — generates embeddings that are used to represent the article content
  - ArticleRepository — provides the collection of articles being analyzed
  - Article — the articles being compared
- **Important Methods:**
  - analyze()
  - findAll()
  - embed()
- **Execution:** The NewsAnalysisAgent selects the Clustering strategy to identify the articles that are highly similar. This strategy first retrieves the relevant articles through the ArticleRepository.findAll() method. Each article is then converted into an embedding using LLMClient.embed(). The resulting embeddings are compared to determine which articles are highly similar to each other to be considered as duplicate stories.

F06 — Event Clustering:

- **Related Use Case:** UC04 — Detect Duplicate Stories and Cluster Stories
- **Related Sequence Diagram:** SD02 — Duplicate-Story Detection and Cluster
- **Classes Involved:**

    - NewsAnalysisAgent — coordinates the event clustering analysis
    - AnalysisStrategy — a generic interface for each analysis strategy
    - Clustering — groups the related news articles into event clusters
    - ArticleRepository — provides the articles that will be clustered
    - EventClusterRepository — stores and retrieves the generated event clusters
    - EventCluster — represents a grouped set of articles with the same or related event
    - Article — the articles that are being grouped
- **Important Methods:**
    - analyze()
    - findAll()
    - addArticle()
    - save()
- **Execution:** The NewsAnalysisAgent selects the Clustering strategy through the AnalysisStrategy interface. The strategy will receive a collection of retrieved articles. With these articles, the strategy will analyze each one and group the related stories into EventCluster objects. Articles that belong to the same event are associated with the corresponding cluster through the EventCluster.addArticle() method. The resulting clusters are then stored using the EventClusterRepository.save() method.

F07 — Entity Extraction:

- **Related Use Case:** UC03 — Analyze Articles
- **Related Sequence Diagram:** SD01 — Article Retrieval and Analyzation
- **Classes Involved:**
    - AgentController — provides the interface for requesting AI analysis
    - NewsAnalysisAgent — coordinates entity extraction and selects the appropriate strategy
    - AnalysisStrategy — a generic interface for each analysis strategy
    - EntityExtraction — the strategy that actually extracts the entities from an article
    - LLMClient — provides access to the language model used for extraction
    - Entity — represents an extracted entity (person, organization, location, etc.)
    - Article — the article content where entities are being extracted from
- **Important Methods:**
    - analyzeArticle()
    - analyze()
    - generate()
- **Execution:** The NewsAnalysisAgent selects the EntityExtraction strategy through the AnalysisStrategy interface when entity information is required from an article. The strategy will send the article contents to LLMClient.generate() which uses the language model to identify entities. The extracted information is represented using Entity objects associated with the article.

F08 — Timeline Generation:

- **Related Use Case:** UC05 — Generate Timeline
- **Related Sequence Diagram:** SD05 — Timeline Generation
- **Classes Involved:**
    - TimelineView — displays the generated timeline to the user
    - AgentController — provides the application-level operation for requesting a timeline
    - NewsAnalysisAgent — coordinates the analysis required to make the timeline

    - AnalysisStrategy — a generic interface for each analysis strategy
    - Clustering — groups the related news articles into event clusters
    - EventClusterRepository — stores and retrieves the generated event clusters
    - TimelineGenerator — converts and assembles the event clusters into a chronological timeline
    - Timeline — represents the final generated collection of timeline events
    - TimelineEvent — represents a single event and its date within the timeline
- **Important Methods:**
    - generateTimeline()
    - findAll()
    - analyze()
    - save()
    - sortByDate()
    - addEvent()
- **Execution:** When the user requests a timeline, TimelineView will send the request through AgentController to the NewsAnalysisAgent. The system will retrieve the relevant articles and use the Clustering strategy through the AnalysisStrategy interface. The strategy will organize related articles into EventCluster objects, and the resulting clusters can be stored or retrieved through EventClusterRepository. The TimelineGenerator will then convert the event clusters into chronological TimelineEvent objects and add them to a Timeline using the Timeline.addEvent() method. The events are ordered using the sortByDate() method, and the completed timeline is returned back to the TimelineView for display.

F09 — Multiple Source Comparison:

- **Related Use Case:** UC06 — Compare News Sources
- **Related Sequence Diagram:** SD03 — Multiple Source Comparison
- **Classes Involved:**
    - ComparisonPanel — allows the user to view and select the articles being compared
    - AgentController — provides the application-level interface for requesting source comparison
    - NewsAnalysisAgent — coordinates the comparison process
    - AnalysisStrategy — a generic interface for each analysis strategy
    - Comparison — the strategy that compares the article content from multiple sources
    - ClaimExtraction — the strategy that extracts claims that can be used during source comparison
    - LLMClient — provides language model processing for comparison and claim analysis
    - SourceLinkManager — provides links back to the original source articles
    - Article — the articles being compared
- **Important Methods:**
    - compareSources()
    - analyze()
    - generate()
    - getSourceLink()
- **Execution:** The user selects multiple articles through the ComparisonPanel which will be sent for analysis. The NewsAnalysisAgent uses the appropriate analysis strategies to examine the articles. The ClaimExtraction strategy will first identify the claims made by each source. And the Comparison strategy will then analyze the selected articles and compare their coverage. The LLMClient.generate() method is called to identify the similarities, differences, and variations in reporting, and these results are returned back through the agent.

As well, the SourceLinkManager will provide the original source links so that the comparison results can be traced back to the corresponding articles.

F10 — Claim Extraction:

- **Related Use Case:** UC06 — Compare News Sources
- **Related Sequence Diagram:** SD03 — Multiple Source Comparison
- **Classes Involved:**
  - NewsAnalysisAgent — coordinates the claim-extraction process
  - AnalysisStrategy — a generic interface for each analysis strategy
  - ClaimExtraction — the strategy that extracts claims from an articles content
  - LLMClient — provides language model processing for extracting the claims
  - Claim — represents an extracted claim along with its associated information
  - Article — the article content where claims are being extracted from
- **Important Methods:**
  - analyze()
  - generate()
- **Execution:** The NewsAnalysisAgent selects the ClaimExtraction strategy through the AnalysisStrategy interface when claims need to be identified from an article. The strategy will send the article's content to LLMClient.generate() which will identify any meaningful claims. Each extracted claim is represented by a Claim object containing information such as its text and confidence value. The extracted claims can then be used by the multiple-source comparison feature.

F11 — Source Linking:

- **Related Use Case:** UC06 — Compare News Sources
- **Related Sequence Diagram:** SD03 — Multiple Source Comparison
- **Classes Involved:**
  - ComparisonPanel — displays the source information and links as well as the comparison results
  - SourceLinkingManager — manages and retrieves the source URL associated with an article
  - Article — the article that is associated with the original source
- **Important Methods:**
  - linkSource()
  - getSourceLink()
- **Execution:** The SourceLinkManager maintains the relationship between an Article and its source URL. When the comparison results are displayed, the ComparisonPanel can request the source link through the SourceLinkManager.getSourceLink() method.

F12 — Topic Tracking:

- **Related Use Case:** UC07 — Track Topics and Feed
- **Related Sequence Diagram:** SD04 — Topic Tracking and Feed Updates
- **Classes Involved:**
  - TopicController — handles user requests to create, update, and manage topics
  - TopicRepository — stores and retrieves the topic information
  - Topic — represents the tracked topic
  - TopicTracker — monitors the tracked topics and triggers refreshes
  - NewsController — coordinates retrieval of news articles for the tracked topics
  - NewsSourceFactory — creates the appropriate source adapters based on the selected provider

    - NewsSourceAdapter — common interface for all news source adapters
    - ArticleRepository — stores the newly retrieved articles
    - NewsDashboard — refreshes to display the updated topic and article information to the user
- **Important Methods:**
    - createTopic()
    - updateTopic()
    - deleteTopic()
    - trackTopics()
    - refresh()
    - retrieveArticles()
    - createAdapter()
    - fetchArticles()
    - save()
- **Execution:** The system maintains the user's tracked topics through TopicTracker. The TopicController manages the topic information using TopicRepository, while the TopicTracker.trackTopics() method maintains the topics that should be monitored. When tracked topics are refreshed, the tracker works with NewsController to retrieve new articles. NewsController obtains the appropriate source adapter through NewsSourceFactory, and the adapter then retrieves the articles from the external provider. These retrieved articles are stored through ArticleRepository. TopicTracker can notify the relevant UI component when updated articles are available so that the dashboard can refresh.

F13 — Personalized Feed:

- **Related Use Case:** UC07 — Track Topics and Feed
- **Related Sequence Diagram:** SD04 — Topic Tracking and Feed Updates
- **Classes Involved:**
    - FeedController — handles the requests for a personalized feed
    - FeedGenerator — generates and refreshes the user's personalized feed
    - ArticleRepository — provides the saved articles
    - TopicRepository — provides the tracked topics
    - SearchHistoryRepository — provides the user's recent searches
    - NewsDashboard — displays the personalized feed
- **Important Methods:**
    - getPersonalizedFeed()
    - generateFeed()
    - refreshFeed()
    - findAll()
- **Execution:** When the user requests a personalized feed, FeedController will send a feed request to FeedGenerator. The generator will then obtain the user's relevant topics, available articles, and search history to determine their recent interests. FeedGenerator.generateFeed(userId) combines this information to select and organize the articles that are relevant to the user. refreshFeed() will allow the feed to be regenerated when new articles are available.

F14 — Saved Articles:

- **Related Use Case:** UC08 — Save Articles
- **Related Sequence Diagram:** SD07 — Saved Articles

- **Classes Involved:**
  - ArticleViewer — displays an article and the save/remove actions
  - ArticleRepository — stores and retrieves the user's selected articles
  - Article — the article that's being saved or removed
- **Important Methods:**
  - saveArticle()
  - save()
  - removeSaved()
  - findById()
  - findSaved()
- **Execution:** The user saves an article through the ArticleViewer. The article can be retrieved through ArticleRepository.findById() and marked as saved using the Article.save() method. The updated article is then stored through ArticleRepository.save(). When the user requests their saved articles, ArticleRepository.findSaved() will return the articles whose saved state is active. The article can also be removed from the saved collection by the user through the Article.removeSaved() method.

F15 — Natural-Language News Queries:

- **Related Use Case:** UC09 — Natural-Language News Query
- **Related Sequence Diagram:** SD06 — Natural-Language News Queries
- **Classes Involved:**
  - NewsDashboard — provides the GUI for the user to enter and submit a natural-language query
  - NewsCLI — provides a command-line interface for submitting the queries
  - AgentController — provides a simplified interface to the AI agent subsystem
  - NewsAnalysisAgent — coordinates the reasoning, planning, tool use, and response generation
  - PlanningEngine — determines the sequence of steps as well as tools that are needed
  - ToolManager — manages and executes the required tools selected by the agent
  - NewsController — provides the news retrieval functionality that can be used as an agent tool
  - ArticleRepository — provides access to stored articles as context
  - SearchHistoryRepository — stores the user's queries for future reference
  - LLMClient — provides access to the language model
  - ChatModel — the underlying chat model interface that is being used for the language generation
- **Important Methods:**
  - start()
  - processCommand()
  - answerQuery()
  - analyze()
  - plan()
  - createPlan()
  - executeTool()
  - searchNews()
  - save()
  - generate()
  - chat()

- **Execution:** The user enters and submits a natural-language news question through the GUI/CLI. The request is then passed to AgentController.answerQuery(). The NewsAnalysisAgent will analyze the request and use PlanningEngine.createPlan() to determine the steps and tools needed to answer it. Then, ToolManager.executeTool() will execute the required tools which can include NewsController.searchNews() for retrieving current articles. The results are then returned to the agent which uses LLMClient and ChatModel to synthesize the information into a response.

F16 — Daily Briefing Generation:

- **Related Use Case:** UC10 — Generate Daily Briefing
- **Related Sequence Diagram:** SD08 — Daily Briefing Generation
- **Classes Involved:**
  - NewsDashboard — displays the generated daily briefing to the user
  - AgentController — provides the application-level operation for generating a briefing
  - NewsAnalysisAgent — coordinates the article retrieval, clustering, and summarization
  - ArticleRepository — provides the articles that are used to create the briefing
  - AnalysisStrategy — a generic interface for each analysis strategy
  - Clustering — the strategy that groups related stories into event clusters
  - Summarization — the strategy that condenses each article into a summary
  - LLMClient — provides the language model processing needed for clustering and summarization
  - EventCluster — represents a grouped set of articles with the same or related event
- **Important Methods:**
  - generateBriefing()
  - findAll()
  - analyze()
  - getArticles()
  - generate()
- **Execution:** When the user requests a daily briefing, AgentController.generateBriefing() will ask the NewsAnalysisAgent to gather and analyze the relevant current articles. The agent retrieves the available articles through ArticleRepository.findAll() and uses the Clustering strategy to group the related articles into EventCluster objects. The articles associated with each cluster can then be processed by the Summarization strategy to generate a concise summary of each. The LLMClient.generate() provides the language model processing model required for these two strategies. The agent will then combine the resulting summaries into a briefing and return to NewsDashboard to be displayed.

F17 — Search History:

- **Related Use Case:** UC11
- **Related Sequence Diagram:** SD06
- **Classes Involved:**
  - NewsDashboard — displays the user's previous searches and provides access to history management
  - NewsCLI — provides the command-line access to the search history functionality
  - AgentController — coordinates the query request
  - SearchHistoryRepository — stores, retrieves, and manages past queries made by the user
- **Important Methods:**
  - answerQuery()
  - save()

    - findAll()
    - clear()
- **Execution:** Whenever the user performs a natural-language news query, the query is stored using the SearchHistoryRepository.save() method. The repository provides the findAll() method which retrieves the stored search history. As well, the repository provides the clear() method to remove the stored queries. This information allows the GUI/CLI to display previous searches and allow the user to clear them.